Albert0i / mariadb-vss-chinese

Code Issues Pull requests Actions Projects Wiki Security

main mariadb-vss-chinese / README.md

Albert0i minor link fix aa99582 · now

309 lines (232 loc) · 12.5 KB

Vector Semantic Search in Chinese using MariaDB

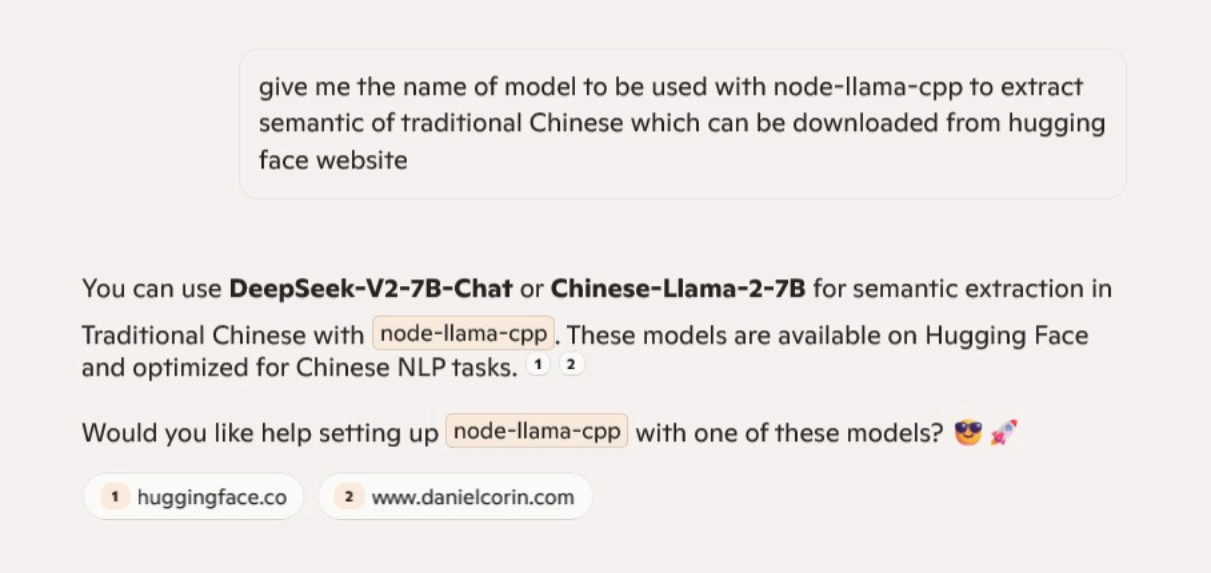
> "the ridiculous confusion which, in some circumstances, can determine the course of a man's life."
> The Castle by Franz Kafka

Prologue

- Vector Semantic Search using MariaDB (Part 1/3)
- Vector Semantic Search using MariaDB (Part 2/3)
- Vector Semantic Search using MariaDB (Part 3/3)

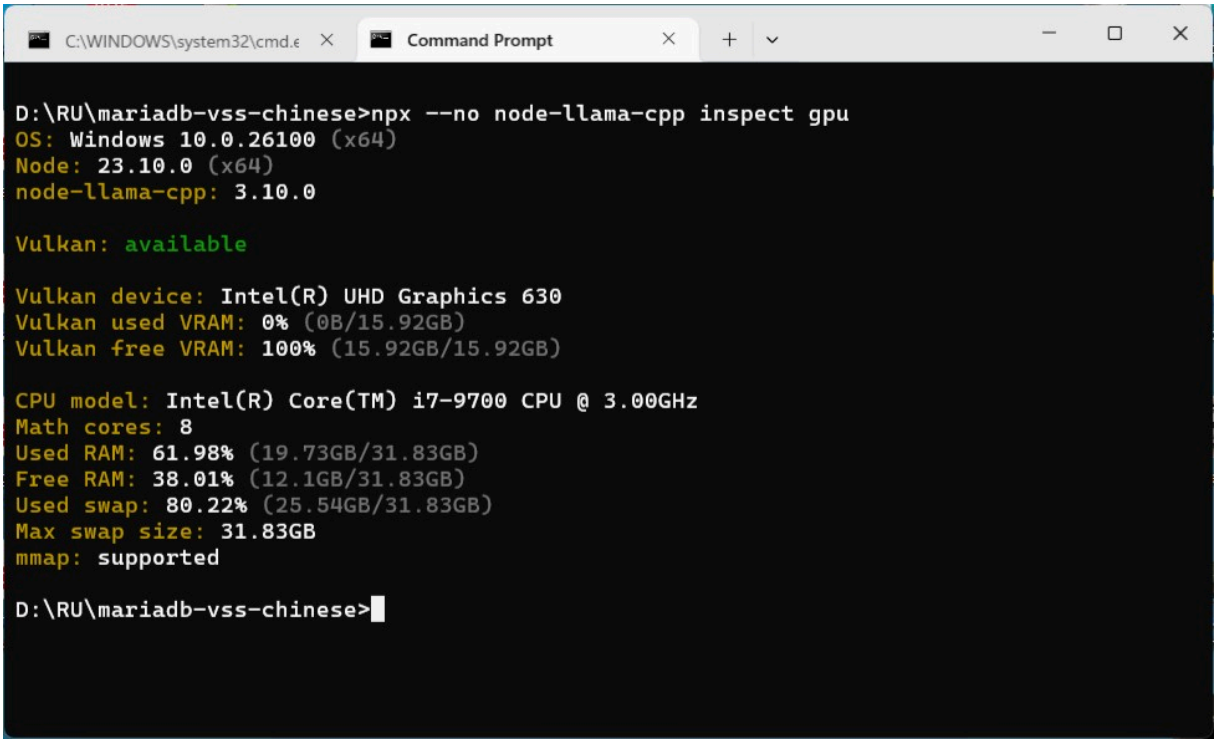
I. Choosing a Model

Previously we used VSS on English data. *Every tool would render useless if it can not work on one's native language*. We need to do the same thing on Chinese data, typically traditional Chinese. To be able to extract semantic, we need a different model as it is suggested by AI.



I head into Hugging Face website and search for Chinese-Llama-2-7B model and inspect my computer with:

```
npx --no node-llama-cpp inspect gpu
```

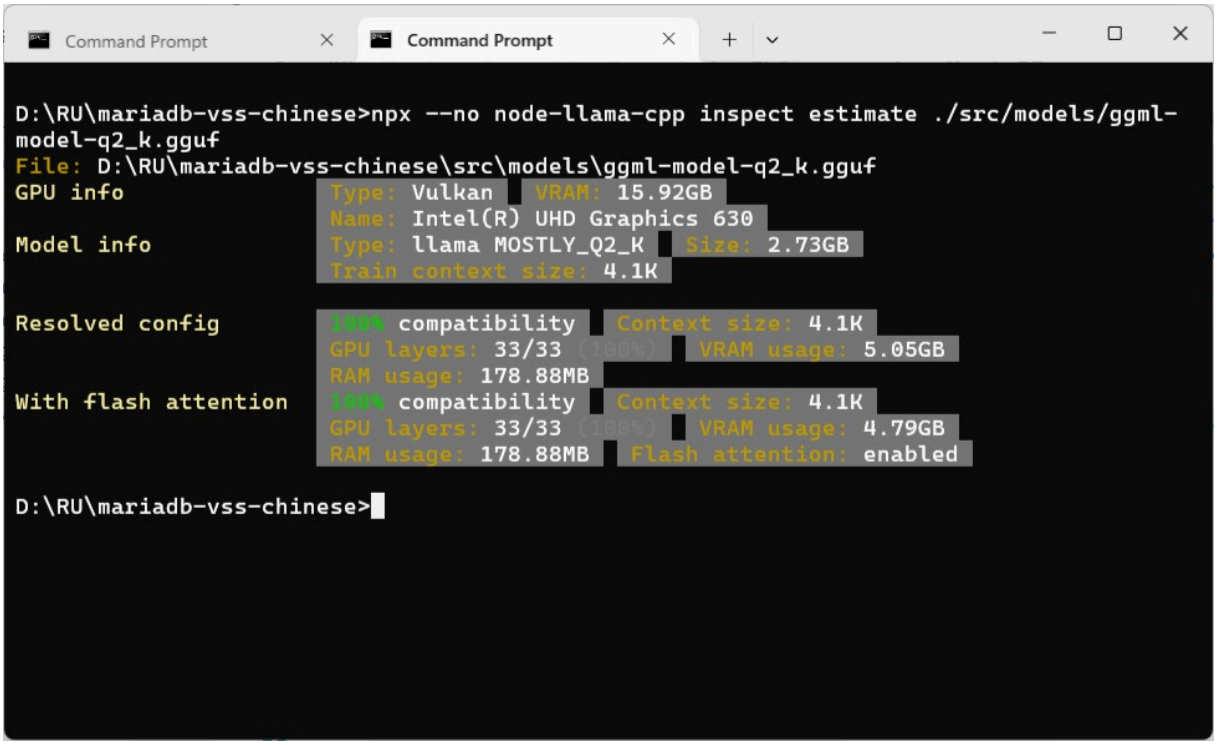


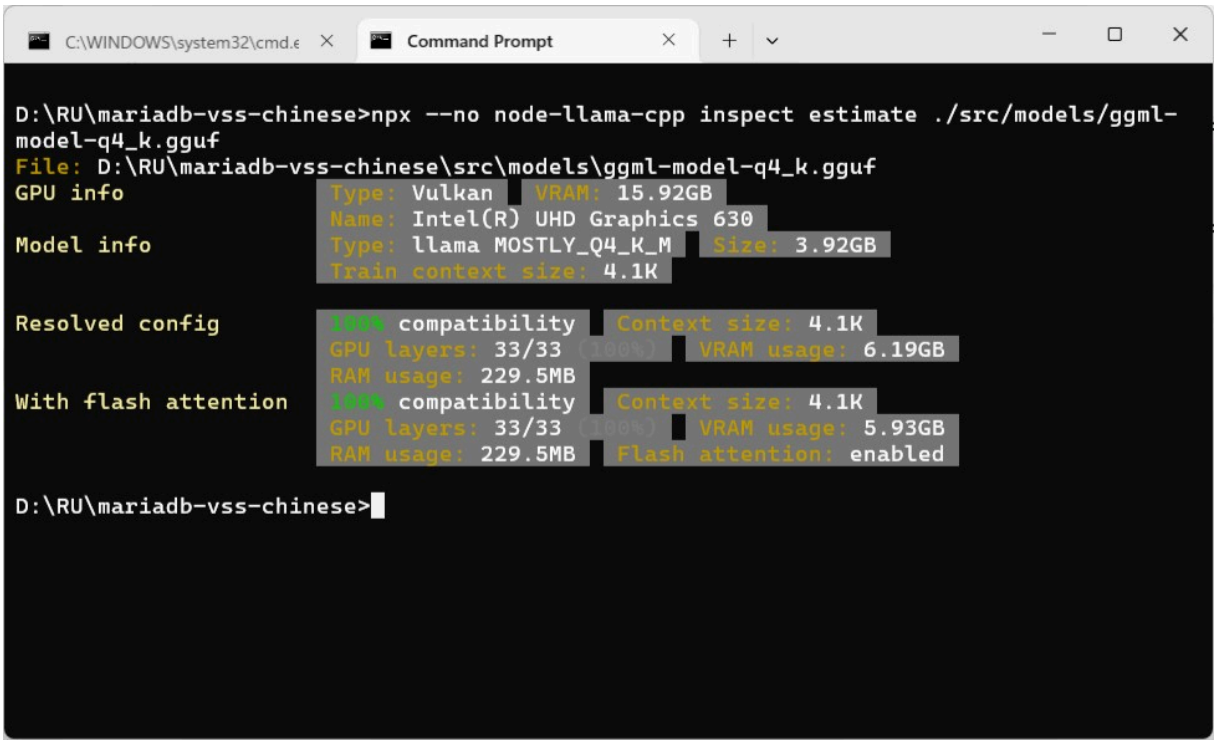
> If the machine you plan to run this model on doesn't have a GPU, you'd probably want to use a small model that can run on a CPU with decent performance.

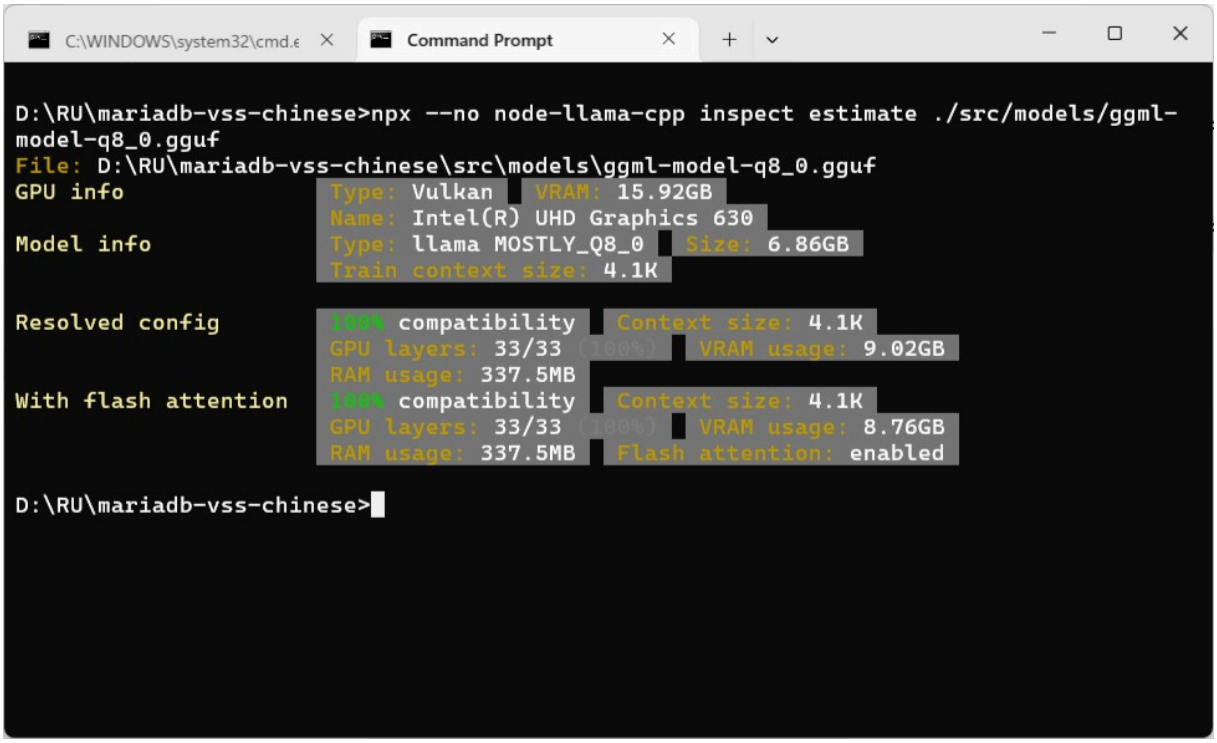
> If you have a GPU, the amount of VRAM you have will determine the size of the model you can run. Ideally, you'd want to fit the entire model in the VRAM to use only the GPU and achieve maximum performance. If the model requires more memory than the available VRAM, parts of it will be offloaded to the RAM and be evaluated using the CPU, significantly reducing the efficiency and speed of inference.

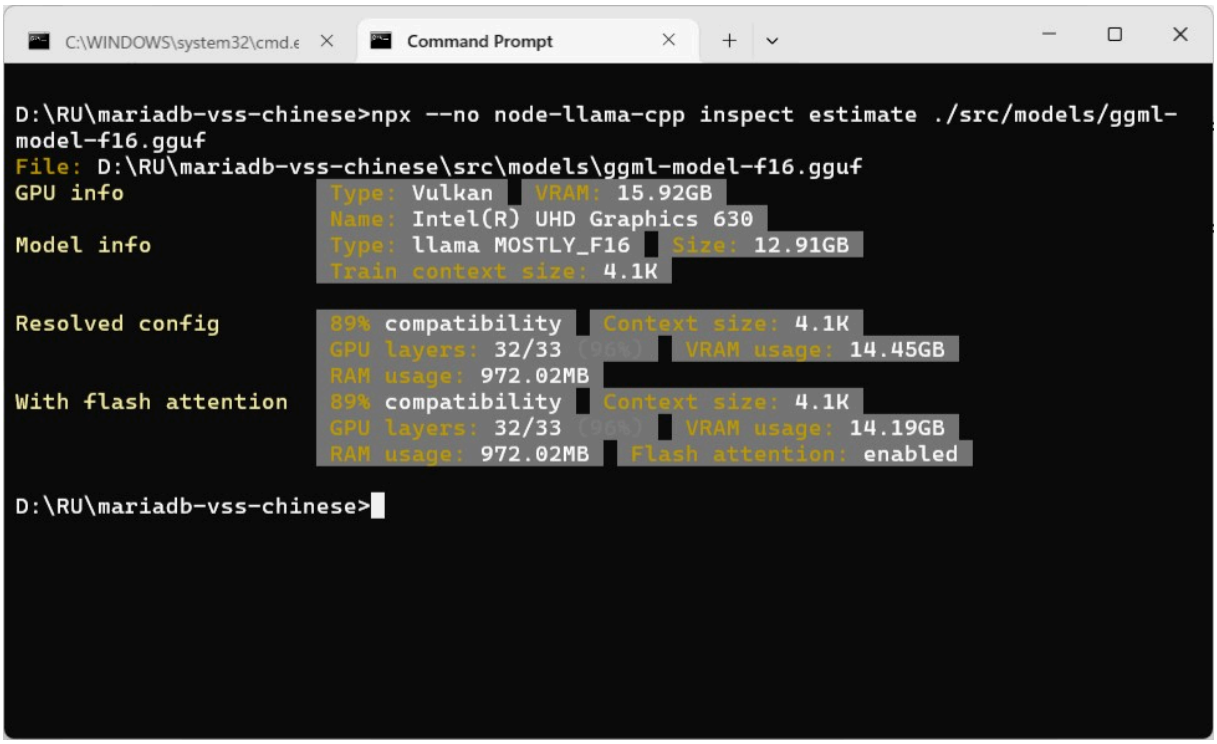
To get a more accurate estimation of how well a model will run:

```
npx --no node-llama-cpp inspect estimate ./src/models/ggml-model-q2_k.gguf
```




C:\WINDOWS\system32\cmd.e
Command Prompt
D:\RU\mariadb-vss-chinese>npx --no node-llama-cpp inspect estimate ./src/models/ggml-model-q4_k.gguf
File: D:\RU\mariadb-vss-chinese\src\models\ggml-model-q4_k.gguf
GPU info
Type: Vulkan VRAM: 15.92GB
Name: Intel(R) UHD Graphics 630
Model info
Type: llama MOSTLY_Q4_K_M Size: 3.92GB
Train context size: 4.1K
Resolved config
100% compatibility Context size: 4.1K
GPU layers: 33/33 (100%) VRAM usage: 6.19GB
RAM usage: 229.5MB
With flash attention
100% compatibility Context size: 4.1K
GPU layers: 33/33 (100%) VRAM usage: 5.93GB
RAM usage: 229.5MB Flash attention: enabled
D:\RU\mariadb-vss-chinese>


C:\WINDOWS\system32\cmd.e
Command Prompt
D:\RU\mariadb-vss-chinese>npx --no node-llama-cpp inspect estimate ./src/models/ggml-model-q8_0.gguf
File: D:\RU\mariadb-vss-chinese\src\models\ggml-model-q8_0.gguf
GPU info
Type: Vulkan VRAM: 15.92GB
Name: Intel(R) UHD Graphics 630
Model info
Type: llama MOSTLY_Q8_0 Size: 6.86GB
Train context size: 4.1K
Resolved config
100% compatibility Context size: 4.1K
GPU layers: 33/33 (100%) VRAM usage: 9.02GB
RAM usage: 337.5MB
With flash attention
100% compatibility Context size: 4.1K
GPU layers: 33/33 (100%) VRAM usage: 8.76GB
RAM usage: 337.5MB Flash attention: enabled
D:\RU\mariadb-vss-chinese>



Suggestion from AI

> Your GPU **can handle models up to around 15GB VRAM usage**, but for **efficient performance**, models should **stay below 80% of total VRAM** (~12GB). If you want **bigger models**, you may need a GPU with more memory or offload layers to **CPU RAM (slower but possible)**.

1. ggml-model-q2_k.gguf (2.73G)
2. ggml-model-q4_k.gguf (3.92G)
3. ggml-model-q8_0.gguf (6.85G)
4. ggml-model-f16.gguf (12.9G)

To strike a balance between speed and accuracy, I opt `ggml-model-q4_k.gguf` (3.92G) instead of `ggml-model-q8_0.gguf` (6.85G). A short comparison is in here. Both of them generate vector of 4096 dimensions.

I have the AI generated 1,200 sentences in traditional Chinese and put it in an array for sake of simplicity. A more realistic scenario would load the data from Oracle or by parsing a CSV file, for example.

It is *crucial* to choose the right model in the first place, change of model involves re-creating all vector embeddings may take hours or even days.

II. Using Embedding

First of all, create a table in target database:

```
USE vss;
```

```
CREATE TABLE documents (
  id          INT AUTO_INCREMENT PRIMARY KEY,

  textChi     VARCHAR(512) NOT NULL,
  visited     INT DEFAULT 0,
  embedding   VECTOR(4096) NOT NULL,

  createdAt   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updatedAt   TIMESTAMP,
  updateIdent INT DEFAULT 0,

  UNIQUE (textChi),
  VECTOR INDEX (embedding) M=16 DISTANCE=cosine
);
```

Note:

1. Number of dimensions is determined by model output;
2. Only one vector index per table and can not be NULL;
3. `M` — Larger values mean slower SELECTs and INSERTs, larger index size and higher memory consumption but more accurate results. The valid range is from `3` to `200`.
4. `DISTANCE` — Distance function to build the vector index for. Searches using a different distance function will not be able to use a vector index. Valid values are `cosine` and `euclidean` (the default).

Next, install Prisma, pull in model and generate client code.

```
model documents {
  id          Int                          @id
@default(autoincrement())
  textChi     String                       @unique(map: "textChi")
@db.VarChar(512)
  visited     Int?                         @default(0)
  embedding   Unsupported("vector(4096)")
  createdAt   DateTime?                    @default(now())
@db.Timestamp(0)
  updatedAt   DateTime?                    @db.Timestamp(0)
  updateIdent Int?                         @default(0)

  @@index([embedding], map: "embedding")
}
```

The code template to generate vector embedding is available here:

```
import {fileURLToPath} from "url";
import path from "path";
import { getLlama } from 'node-llama-cpp';
```

```
const __dirname = path.dirname(fileURLToPath(import.meta.url));

const llama = await getLlama();
const model = await llama.loadModel({
    modelPath: path.join(__dirname, "models", "bge-small-en-v1.5-
q8_0.gguf"),
    eosTokenId: 128001 // Adjust based on tokenizer config
  });
const context = await model.createEmbeddingContext();

async function embedDocuments(documents) {
    const embeddings = new Map();

    await Promise.all(
        documents.map(async (document) => {
            const embedding = await context.getEmbeddingFor(document);

            embeddings.set(document, embedding);

            console.debug(
                `${embeddings.size}/${documents.length} documents
embedded`
            );
        })
    );

    return embeddings;
}

function findSimilarDocuments(embedding, documentEmbeddings) {
    const similarities = new Map();
    for (const [otherDocument, otherDocumentEmbedding] of
documentEmbeddings)
        similarities.set(
            otherDocument,

embedding.calculateCosineSimilarity(otherDocumentEmbedding)
        );

    return Array.from(similarities.keys())
        .sort((a, b) => similarities.get(b) - similarities.get(a));
}

const documentEmbeddings = await embedDocuments([
     "The sky is clear and blue today",
     "I love eating pizza with extra cheese",
     "Dogs love to play fetch with their owners",
     "The capital of France is Paris",
     "Drinking water is important for staying hydrated",
     "Mount Everest is the tallest mountain in the world",
     "A warm cup of tea is perfect for a cold winter day",
     "Painting is a form of creative expression",
```

```
     "Not all the things that shine are made of gold",
     "Cleaning the house is a good way to keep it tidy"
 ]);

const query = "What is the tallest mountain on Earth?";

const queryEmbedding = await context.getEmbeddingFor(query);

const similarDocuments = findSimilarDocuments(
    queryEmbedding,
    documentEmbeddings
);
const topSimilarDocument = similarDocuments[0];

console.log("query:", query);
console.log("Document:", topSimilarDocument);
```

> This example will produce this output:

```
query: What is the tallest mountain on Earth?
Document: Mount Everest is the tallest mountain in the world
```

> This example uses bge-small-en-v1.5

Which is a good start!

III. Seeding

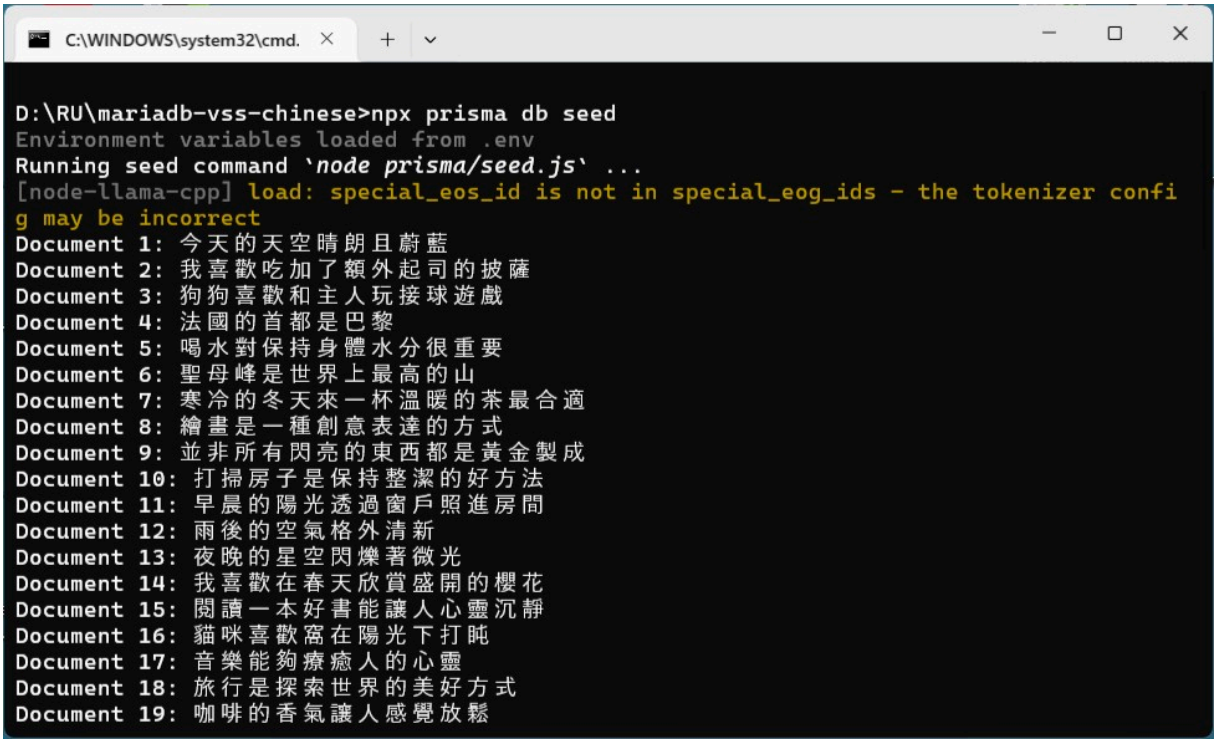
A cheap trick is used to implement `UPSERT` in MariaDB, which is described in INSERT ON DUPLICATE KEY UPDATE. An `UPSERT` prevents from inserting duplicated entry and let's keep track of the duplication.

```
export async function addDocument(document) {
    const { vector } = await
context.getEmbeddingFor(removeWords(document));

    // Add new document
    return await prisma.$executeRaw`
                    INSERT INTO documents (textChi, embedding)
                    VALUES( ${document},
VEC_FromText(${JSON.stringify(vector)}) )
                    ON DUPLICATE KEY
                    UPDATE updateIdent = updateIdent + 1;
                `;
}
```

Run command to seed database:

```
npx prisma db seed
```



Depending on the machine, it may take hours or even days... AFter that verify if there is duplicated entry with:

```
SELECT * FROM documents WHERE updateIdent <> 0;
```

IV. Finding the documetns

Most of the code remains the same, we only add update to `visited` field so that we can check to see search results.

```
export async function findSimilarDocuments(document, limit = 3) {
    const { vector } = await
context.getEmbeddingFor(removeWords(document));

    // Find similar documents
    const docs = await prisma.$queryRaw`
                          SELECT textChi,
                                 VEC_DISTANCE_COSINE(
                                    embedding,

VEC_FromText(${JSON.stringify(vector)})
                                 ) AS distance,
                                 id
                          FROM documents
                          ORDER BY 2 ASC
                          LIMIT ${limit} OFFSET 0;
                        `;
```

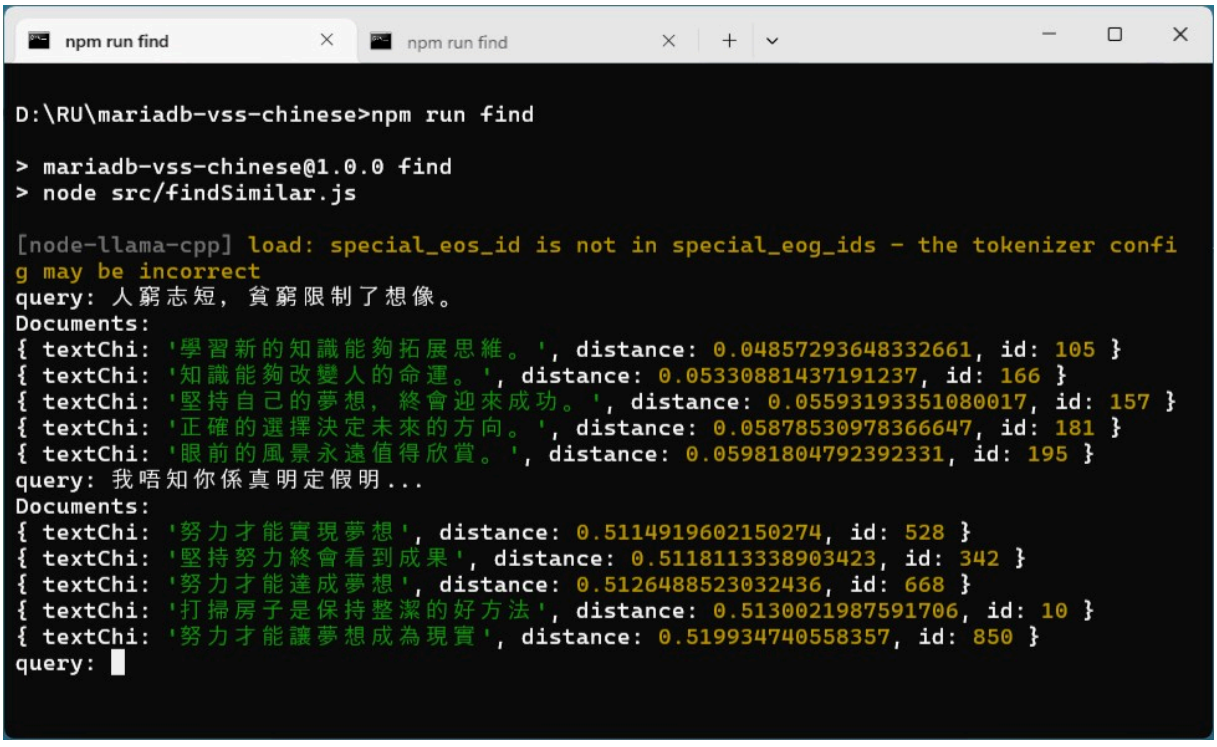
```
    // Update `visited` field
    const promises = [];    // Collect promises
    docs.forEach(doc => {
            promises.push(prisma.$executeRaw`
                            UPDATE documents
                            SET visited = visited + 1,
                                updatedAt = Now(),
                                updateIdent = updateIdent + 1
                            WHERE id=${doc.id}
                        `

            )
        })
    await Promise.all(promises); // Resolve all at once

    return docs
}
```

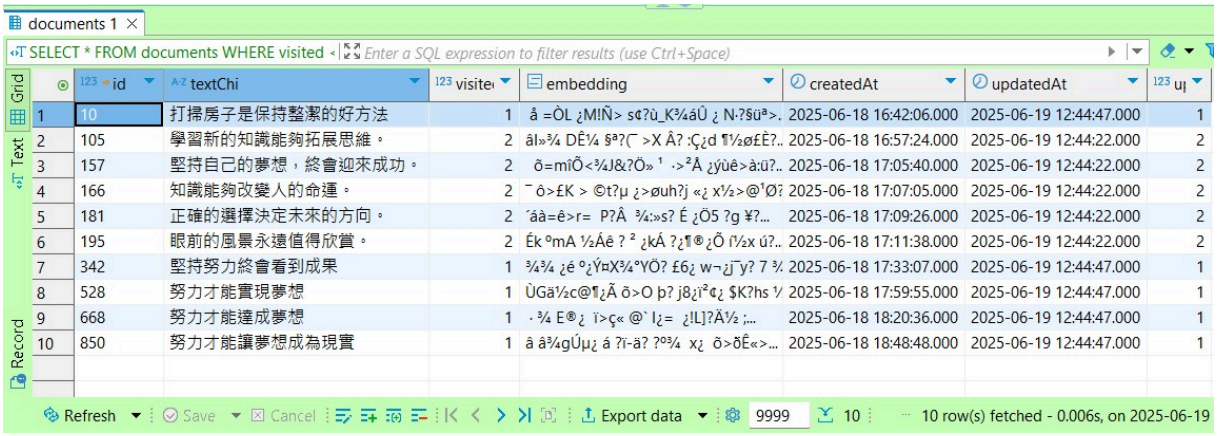
Run command to find the documents:

```
npm run find
```



Verify search results with:

```
SELECT * FROM documents WHERE visited <> 0;
```



V. Building a web site

With a little bit web knowledge and the help of AI, one can quickly build a decent web site, in our case about 95% of code is written by HIM, only the implementation of API route is written by myself. We expose six pages:

1. http://localhost:3000/
2. http://localhost:3000/search
3. http://localhost:3000/add
4. http://localhost:3000/stats
5. http://localhost:3000/details
6. http://localhost:3000/feature

And four API end-points:

1. http://localhost:3000/api/v1/search
2. http://localhost:3000/api/v1/add
3. http://localhost:3000/api/v1/stats
4. http://localhost:3000/api/v1/details

Start the server:

```
npm run dev
```

VI. Last suggestion from AI

For **semantic search in Traditional Chinese**, the current top performer is **BAAI's bge-m3** model. It consistently ranks at or near the top of multilingual benchmarks like MTEB, and it's specifically designed for high-quality sentence embeddings across 100+ languages—including zh-Hant (繁體中文).

🧠 Why bge-m3 stands out

- **Fast and efficient**: Available in quantized GGUF format for local use
- **Proven performance**: Outperforms many older models like MiniLM or E5 on Chinese benchmarks
- You can grab the GGUF version here: bbvch-ai/bge-m3-GGUF

VII. Bibliography

1. Choosing a Model
2. Using Embedding
3. Raw queries
4. CRUD
5. The Castle by Franz Kafka

Epilogue

> "I am certainly ignorant, but facts are facts, which is very sad for me but also advantageous, since an ignorant man will dare to do more, so I will happily go about in my ignorance with what I am sure are its unfortunate consequences for a little longer, as long as my strength allows. "
> The Castle by Franz Kafka

EOF (2025/06/27)